

How Effective Are NPM Malicious Package Detectors? A Large-Scale Empirical Study

Wenbo Guo
Nanyang Technological University
Singapore, Singapore
wenbo002@e.ntu.edu.sg

Zhongwen Chen
Sichuan University
Chengdu, China
yinglilikz@stu.scu.edu.cn

Zhengzi Xu*
Imperial Global Singapore
Singapore, Singapore
z.xu@imperial.ac.uk

Chengwei Liu
Nankai University
Tianjin, China
chengwei.liu@nankai.edu.cn

Ming Kang
Sichuan University
Chengdu, China
kreasem@gmail.com

Shiwen Song
Singapore Management University
Singapore, Singapore
swsong@smu.edu.sg

Chengyue Liu
Nanyang Technological University
Singapore, Singapore
CHENGYUE001@e.ntu.edu.sg

Yijia Xu
Sichuan University
Chengdu, China
xuyijia@scu.edu.cn

Weisong Sun
Nanyang Technological University
Singapore, Singapore
weisong.sun@ntu.edu.sg

Yang Liu
Nanyang Technological University
Singapore, Singapore
yangliu@ntu.edu.sg

Abstract

The NPM ecosystem faces escalating threats from malicious packages that exploit its open publication model. While numerous detection tools have been proposed, they are evaluated on disparate datasets with inconsistent settings, making cross-tool comparison unreliable and leaving practitioners without clear guidance.

We present the first large-scale empirical study of NPM malicious package detection, evaluating 11 tools with 16 variants on a unified benchmark of 6,420 malicious and 7,288 benign packages annotated with 11 behavior categories and 8 evasion techniques. Unlike prior work, we inspect each tool's source code to explain why tools succeed or fail, not merely how often. Our key findings: (1) the precision and recall a tool achieves are structurally determined by how it resolves the ambiguity between code capability and malicious intent, with IntelGuard reaching the best F1 at 95.98% by grounding its judgment in retrieved evidence and GuardDog the best among conventional tools at 93.32%; (2) behavioral coupling amplifies detection signals when behaviors co-occur, raising SAP_DT from 3.2% to 79.3% for the collect-and-exfiltrate chain; (3) 80.3% of malware uses no evasion because the ecosystem lacks mandatory pre-publication scanning; (4) ML degradation is driven by concept convergence rather than concept drift, since malware became simpler and every decision boundary fitted to a corpus ages with it; (5) combination effectiveness equals complementarity

minus false-positive introduction, not paradigm diversity. Strategic combinations reach up to 97.21% accuracy and 97.02% F1. We release our benchmark and evaluation framework.

CCS Concepts

• Security and privacy → Malware and its mitigation.

Keywords

Software supply chain security, Malicious package detection, NPM ecosystem, Empirical study, Benchmark

ACM Reference Format:

Wenbo Guo, Zhongwen Chen, Zhengzi Xu, Chengwei Liu, Ming Kang, Shiwen Song, Chengyue Liu, Yijia Xu, Weisong Sun, and Yang Liu. 2026. How Effective Are NPM Malicious Package Detectors? A Large-Scale Empirical Study. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3837427>

1 Introduction

NPM serves as the cornerstone of JavaScript development, hosting over 3.57 million packages with 7.09 billion weekly downloads [1, 41]. Modern web applications routinely incorporate hundreds of NPM dependencies [5, 44], creating supply chain networks where a single compromised package can affect thousands of downstream projects. The NPM registry adopts an open publication model with minimal gatekeeping [5, 11], which prioritizes accessibility but also enables attackers to publish malicious packages. Reported threats include credential harvesters [39], cryptocurrency miners [9], and backdoor installation scripts [22]. Attackers also exploit naming similarities through typosquatting [56], or gain

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3837427>

access via dependency confusion and compromised maintainer accounts [51]. Several high-profile incidents illustrate the impact of these attacks. The *event-stream* package, with 2 million weekly downloads, was compromised to steal Bitcoin wallet keys [21]. Attacks on *ua-parser-js* [7], *coa*, and *rc* [4] affected thousands of projects, showing that NPM has become a major vector for software supply chain attacks.

To counter these threats, researchers and practitioners have developed detection tools spanning four paradigms, namely static analysis of code and metadata [8, 31], dynamic analysis in a sandbox [9, 35], machine learning over package features [40, 52], and large language models reasoning over code semantics, either from the model’s own knowledge [48, 50] or from evidence retrieved from outside it [15]. Each category has shown promising detection results in their published evaluations.

However, malicious packages remain a persistent problem despite the availability of these tools. More fundamentally, we lack understanding of *why* different detection approaches succeed or fail, which prevents principled tool design and deployment. Four questions remain unanswered.

(*Research Gap 1: Detection performance and its structural causes.*) Existing tools exhibit precision ranging from 50% to 99% and recall from 44% to 99%. Since malicious and benign packages invoke the same OS APIs (e.g., *https.request*, *child_process.exec*), every detection tool must resolve the ambiguity between *capability* and *intent*. Capability is what an API makes possible, intent is the purpose it serves, and the *capability-intent gap* is the distance between them that no call site reveals, since *child_process.exec* drives a build script and a reverse shell alike. No study has traced the precision and recall a tool achieves back to how its design resolves this gap.

(*Research Gap 2: Attack-detection interaction.*) Individual tool papers report overall accuracy on their own datasets, but the relationship between specific attack characteristics (behaviors, evasion techniques, attack surface) and detection difficulty remains unclear. It is unknown whether detection failures stem from sophisticated evasion or from a mismatch between where the attack resides (e.g., package.json vs. JavaScript files) and what the tool analyzes.

(*Research Gap 3: Temporal degradation mechanisms.*) ML-based tools trained on historical data are known to suffer performance degradation, commonly attributed to “concept drift.” However, the specific mechanism driving degradation in the npm ecosystem has not been established. It is unclear whether attackers develop more sophisticated evasion over time or whether changes in malware coding style, account for the observed decline.

(*Research Gap 4: Combination principles.*) Practitioners are advised to deploy multiple tools for better coverage, yet no study has examined the principles governing combination effectiveness. It is unknown whether tool diversity guarantees improvement or whether certain combinations can be counterproductive.

To address these gaps, we conduct a large-scale empirical study of NPM malicious package detection tools, combining quantitative evaluation with source-code analysis of each tool’s detection mechanism. We curate a unified benchmark of 6,420 malicious and 7,288 benign packages, annotated with 11 behavior categories and 8 evasion techniques. Our study investigates four research questions:

- **RQ1 (Detection Effectiveness):** What is the detection performance of current tools, and what structural mechanisms determine the precision and recall each tool achieves? (*Gap 1*)
- **RQ2 (Fine-grained Behavioral Analysis):** How do malicious behaviors, evasion techniques, and attack surfaces interact with different detection approaches? (*Gap 2*)
- **RQ3 (Temporal Evolution Analysis):** How does detection performance evolve over time, and what mechanism drives the temporal degradation of ML-based tools? (*Gap 3*)
- **RQ4 (Tool Complementarity Analysis):** What principles govern the effectiveness of tool combinations, and under what conditions can combination be counterproductive? (*Gap 4*)

To answer these questions, we go beyond running tools as black boxes: we inspect the source code of all 11 tools to understand their detection rules, feature designs, thresholds, and architectural assumptions. This enables us to explain *why* tools succeed or fail. We construct the benchmark from academic datasets and security advisories, build its taxonomy through LLM-assisted analysis and expert review, and evaluate 11 tools with 16 variants across packages published from 2020 to 2025.

First, what a tool detects is decided by how it resolves the ambiguity between code *capability* and *intent*, since malicious and benign packages invoke identical OS APIs. IntelGuard reaches the best F1 (95.98%) by grounding each judgment in retrieved evidence and GuardDog the best among conventional tools (93.32%) through taint analysis, while the rest span 50.41% to 99.88% precision and 44.41% to 98.69% recall. SocketAI runs the same model as IntelGuard yet trails it by 49 points of recall, because it asks the model to judge the code alone while IntelGuard first retrieves known malware for the model to compare it against (RQ1). Second, among the 8 evasion categories, environment detection (mean 51.7%) and anti-analysis (mean 52.7%) are the hardest, and ML-based tools fare worst of all, with SAP_DT and SAP_RF catching 5% of anti-analysis (RQ2). Third, ML degrades sharply over time, with SAP_DT falling from 87.15% to 39.49% between 2021 and 2023. We trace this to *concept convergence* rather than concept drift, since attackers adopted minimal-footprint code that is statistically indistinguishable from benign packages, so any boundary fitted to earlier malware ages with it (RQ3). Fourth, tool combinations reach 97.21% accuracy and 97.02% F1, above any single tool, though the margin narrows as the best single tool improves. Combination effectiveness depends on complementarity minus false-positive introduction rather than paradigm diversity, since a high-recall, low-precision partner can degrade a strong tool (RQ4).

The main contributions of this paper are:

- (1) **Benchmark and taxonomy.** We curate the largest NPM malware benchmark annotated at the behavior level (6,420 malicious + 7,288 benign packages) with a taxonomy of 11 behavior categories and 8 evasion techniques, enabling fine-grained analysis not possible with prior datasets.
- (2) **Large-scale Evaluation.** We evaluate 11 tools (16 variants) and, unlike prior benchmarks that treat tools as black boxes, inspect each tool’s source code to identify the structural mechanisms behind its performance.

- (3) **Novel findings.** We identify five structural principles: (i) *capability-intent ambiguity* as the root cause of precision-recall trade-offs; (ii) *concept convergence* driving ML temporal degradation; (iii) *attack surface mismatch* leaving 21% of malware structurally undetectable by code-analysis tools; (iv) *behavioral coupling amplification* boosting detection signals by up to 76 percentage points; and (v) *complementarity-minus-FP-introduction* as the governing principle of tool combination effectiveness.
- (4) We release our dataset, evaluation framework, and all analysis scripts to facilitate future research [2].

2 Related Work

2.1 NPM Supply Chain Security

When developers run `npm install`, npm resolves dependencies and automatically executes the `preinstall`, `install`, and `postinstall` lifecycle scripts with full user privileges before any explicit code invocation [11, 51], after which packages may freely access file systems, network resources, and environment variables [9, 24, 34]. Only 2.2% of packages declare such scripts [51], yet what those scripts execute propagates to every dependent project, since an average package implicitly trusts 79 third-party packages [56]. Attackers reach this surface either by inducing a victim to install the wrong package or by subverting one the victim already trusts. Name confusion spans the 13 mechanisms Neupane et al. separate across 1,232 documented attacks, of which typosquatting is only one [32, 42, 46], dependency confusion exploits name resolution by publishing public packages that match private internal names [3, 6, 24, 26], and account takeover requires no new package [10, 13, 56].

Prior npm work either characterizes the attacks without measuring detectors, or measures detectors without explaining them. Backstabber’s Knife Collection organizes malicious packages into attack trees [34], Ladisa et al. generalize such observations into a taxonomy of attack vectors [24], and Zhou et al. relate a large corpus of them through a knowledge graph [55], yet none of the three reports how a detector responds to the attacks it catalogues. Zahan et al. compare two GPT models with CodeQL [50], Wyss et al. test LLMs on version diffs [48], and ProfMal [20] is evaluated against five detectors, but each reports aggregate scores rather than the mechanism that produces them. We evaluate 11 tools across 16 variants covering all four paradigms on identical inputs, label each package with 11 behavior and 8 evasion categories, track detection from 2011 to 2025, and inspect each tool’s source code to explain why it succeeds or fails.

2.2 NPM Malicious Package Detection

Detection tools differ in the evidence they use to resolve the capability-intent ambiguity, and Table 1 groups them into four paradigms. (1) *Static-based* methods analyze the code without executing it. OSS-Gadget [31] and GuardDog [8] match rules and Semgrep patterns against code and metadata, LastJSmile [38] matches regular expressions against phantom files, and GENIE [14], MalWuKong [28], and diff-CodeQL [12] track information flow with CodeQL to trace a sensitive call back to its source. Their evidence is confined to what the text itself expresses. (2) *Dynamic-based* methods execute the

Table 1: Overview of Studied npm Malicious Package Detection Tools

Type	Name	Year	Target	Feature	Classifier	Avail.
Static-Based	LastJSmile [38]	2022	Phantom files	Code diff, Metadata	Regex	✓
	OSSGadget [31]	2024	Full package	Code, Metadata	Rule-based	✓
	GuardDog [8]	2024	Full package	Code, Metadata	Semgrep	✓
	diff-CodeQL [12]	2023	Full package	Code diff	CodeQL	✓
	GENIE [14]	2024	Full package	Code	CodeQL	✓
Dynamic-Based	MalWuKong [28]	2023	Full package	Code, API call	CodeQL	✓
	MalOSS [9]	2021	Source, binaries	API call, Dynamic	Rule-based	✓
	Packj [35]	2023	Full package	API call, Dynamic	Rule-based	✓
	Cerebro [52]	2023	Full package	AST	BERT	✓
	SAP [25]	2023	Full package	Statistical	DT, RF, XGB	✓
ML-Based	AMALPI [40]	2022	Full package	API call, Metadata, Update	DT, NB, SVM	✓
	DONAPI [18]	2024	Full package	AST, API sequence, Dynamic	RF	✓
	Maltracker [49]	2024	Full package	Code, Graph	DT, RF, XGB	✓
	Ohm et al. [33]	2022	Full package	Metadata, Code	SVM, MLP, RF	✓
	MeMPec [17]	2024	Metadata	Metadata	SVM, GLM, GBM, DRF, DL	✓
	Maldet [53]	2024	Full package	Statistical, API call, Metadata	DT, RF, NB, SVM	✓
	SpiderScan [19]	2024	Full package	Code, API call, Dynamic	RF	✓
	MalPacDetector [47]	2025	Full package	AST, Feature Set	RF, SVM, NB, MLP	✓
	ProfMal [20]	2025	Full package	Behavior graph, Dynamic	Graph classifier	✓
	EMPHunter [29]	2025	Install scripts	Script similarity	Clustering	✓
LLM-Based	SocketAI [50]	2025	Full package	Code Content	LLMs	✓
	IntelGuard [15]	2026	Full package	Code, Threat intel.	LLMs, RAG	✓

Note: Avail.=Available. ✓=Yes, ✗=No. DT=Decision Tree, RF=Random Forest, XGB=XGBoost, NB=Naive Bayes, SVM=Support Vector Machine, MLP=Multi-Layer Perceptron.

package and observe its runtime behavior, with MalOSS [9] tracing sensitive API invocations and Packj [35] adding a sandbox to its metadata checks. Execution yields evidence of behavior rather than of appearance, at the cost of running untrusted code. (3) *ML-based* methods fit a decision boundary to a corpus and differ in the features they fit it to, from statistical properties [25] and ASTs [47, 52] to call graphs [49], API sequences [18, 19], and metadata [17, 33, 40, 53]. ProfMal [20] builds a behavior graph for each package, resolving by execution the calls that static analysis cannot, and classifies the completed graph. EMPHunter [29] requires no malicious examples, clustering the installation scripts of new packages and ranking the outliers. The corpus therefore bounds what they can distinguish. (4) *LLM-based* methods prompt a model to interpret the code. SocketAI [50] queries GPT over three rounds and directs the model to challenge its own conclusions, and Wyss et al. [48] apply the same design to version diffs, whereas IntelGuard [15] retrieves precedent, distilling over 8,000 threat intelligence reports into a knowledge base and asking the model whether the code’s behavior matches the package’s stated purpose. Their evidence is either the model’s own knowledge or what a retrieval step supplies. Each paradigm has advanced on its own dataset, experimental setting, and choice of baselines. We compare them on one benchmark and examine why each succeeds or fails.

3 Evaluation Framework

3.1 Dataset Construction

3.1.1 Malicious Dataset Construction. To construct a representative malicious dataset, we systematically collect NPM malware packages from two categories of sources: academic datasets and public security advisories.

Academic Datasets. Table 2 lists eleven released datasets, of which five make their NPM source downloadable together with machine-readable version and year metadata. We collect all five: BKC [34] (2,113 versions), DONAPI [18] (1,159), DataDog [23] (935), MalOSS [9] (567), and Maltracker [49] (230), spanning 2020 to 2024. **Security Advisories.** OSV and Snyk record reported malicious NPM packages by name and version, but the registry removes those packages once they are disclosed. Since removed packages often remain in downstream mirrors [16], we take the names and

Table 2: Dataset Sources

Paper	Count	Vers.	Year	Upd.	Avail.	Paper	Count	Vers.	Year	Upd.	Avail.
Cerebro [52]	1,789	1,789	2023	✗	✗	MalOSS [9]	567	567	2020	✗	✓
SocketAI [50]	2,180	2,776	2024	✗	✗	BKC [34]	2,113	2,113	2020	✓	✓
AMALFI [40]	643	643	2022	✗	✗	Scalco et al. [38]	119	119	2022	✗	✗
SAP [25]	102	102	2023	✗	✗	DataDog [23]	821	935	2024	✓	✓
DONAPI [18]	1,159	1,159	2024	✗	✓	Maltracker [49]	230	230	2024	✗	✓
SpiderScan [19]	364	364	2024	✗	✗	Ours	4,286	6,420	2025	✓	✓

Note: Vers.=Versions, Upd.=Update, Avail.=Available. ✓=Yes, ✗=No.

versions from the advisories and retrieve the source from those mirrors, which recovers 4,547 versions the registry no longer serves.

3.1.2 Benign dataset construction. For benign packages, we collected 7,288 packages from the official NPM registry by monthly download count, taking the latest version of each. Popularity drives the selection for three reasons. Popular packages carry larger and more varied codebases and trigger significantly higher false-positive rates than randomly sampled ones [27, 45], which makes the precision we measure on them a conservative estimate. They are also the least likely to carry undiscovered malware, since a large user base and community scrutiny find it quickly. And they are what developers actually deploy, so a detector’s performance on them reflects the setting it faces in practice. We verified against OSV and Snyk that none of the 7,288 is a known malicious package.

3.2 Dataset Preprocess

Deduplication. We remove 262 duplicate entries from the original 9,551 malicious versions, leaving 9,289. **Placeholder Removal.** When NPM removes a malicious package, it replaces the content with a placeholder version marked `0.0.1-security` [36], and we remove 1,502 such versions, leaving 7,787. **Manual Verification.** Source datasets may contain false positives due to mislabeling or outdated annotations. To ensure ground truth quality, three security experts holding PhDs in software supply chain security independently review each package. Since malicious behavior is almost always tied to sensitive resources such as processes, memory, and data [54], we locate those operations with GuardDog and OSSGadget, and each expert judges intent at those locations rather than reading the whole package. A package is labeled as malicious only when at least two experts agree that it performs a recognizable malicious action. Across all 7,787 reviewed packages, the three-way consensus rate is 97.4%, and the average pairwise Cohen’s κ is 0.78. This process removes 1,367 false positives, requiring approximately 287 expert-hours in total. After preprocessing, we obtain a curated dataset of 6,420 malicious versions across 4,286 unique packages. Table 2 summarizes the final dataset statistics and sources.

3.3 Malicious Behavior Taxonomy

To enable fine-grained analysis of detection capabilities, we construct a taxonomy of malicious behaviors through three steps: malicious code localization, context extraction, and behavior clustering.

Malicious Code Localization. Manually locating malicious code across 6,420 packages is challenging, so we automate this step using GuardDog and OSSGadget, two complementary static analysis tools. GuardDog’s taint-based rules identify 10,059 locations (1.57 per package), focusing on data exfiltration and command execution patterns. OSSGadget’s regex rules identify 27,350 locations (4.26 per package) with broader coverage including encoded strings and dynamic code evaluation. Neither tool suffices alone, since

GuardDog reports no match for 647 packages (10.08%) and OSSGadget none for 542 (8.44%), but together they detect 6,352 packages, leaving only 68 packages (1.06%) requiring manual analysis. For locations flagged by both tools, we retain one instance.

Code Context Extraction. Each GuardDog or OSSGadget report gives a file path, a line number, the rule that fired, and the flagged code itself, which is only 1-3 lines and insufficient to determine intent, since the definitions and call chains that give those lines meaning often lie elsewhere in the file. We therefore use GPT-4.1 with temperature 0 and top-p 0.3 to recover that context. The input is that report paired with the entire source file it points into, and the output is a JSON object whose `malicious_code` field holds the extracted context. We prompt the LLM to extract the variable definitions and assignments the flagged line depends on, the function call chains invoking the malicious operations, and the control flow structures determining execution conditions. The LLM only expands context and never decides whether code is malicious. Ground-truth labels come entirely from the dataset. For the 68 packages evading both tools, security experts perform manual extraction following the same method.

We extract 17,285 code contexts from all detections and obtain 5,699 unique contexts after deduplication. The detection count exceeds the package count because a single package often contains multiple locations, such as an install script in package.json and a payload in a JavaScript file. The unique count is lower than the package count because typosquatting campaigns reuse identical payloads across many packages with only the name changed.

Behavior Summary and Clustering. From each extracted code context, we prompt the LLM to generate a structured behavioral summary that captures (i) the triggering condition/entry point, (ii) the main malicious actions (e.g., collection, execution, exfiltration), and (iii) the key targets or artifacts involved (e.g., tokens, environment variables, files, network endpoints), yielding 5,699 summaries, one per unique context. We encode each summary using Sentence-BERT (all-mpnet-base-v2) [37], apply UMAP for dimensionality reduction, and cluster the embeddings using K-Means [30]. To determine the number of clusters, we sweep K over a candidate range and select the value that jointly optimizes clustering quality (e.g., silhouette score) and stability across random seeds, while yielding semantically coherent and interpretable groups upon manual inspection. Three domain experts (the same annotators as in Section 3.1) independently reviewed the clusters against the original code contexts, assigned a label to each cluster, and proposed merges for functionally overlapping groups. Initial pairwise label agreement reached 84.2%, after which the experts resolved disagreements through discussion. Eleven categories survive this consolidation, so their number comes from the data rather than from a taxonomy fixed in advance. The resulting behaviors include data exfiltration, arbitrary command execution, credential theft, environment reconnaissance, and persistence installation.

Context Extraction Validation. To evaluate the accuracy of our LLM-based code context extraction, we randomly sample 500 extracted contexts and have three security experts independently verify whether each context correctly captures the complete malicious code segment with its surrounding dependencies. A context is marked as correct only when at least two experts agree. The validation achieves 97.2% accuracy (486/500 correct) with a 97.4%

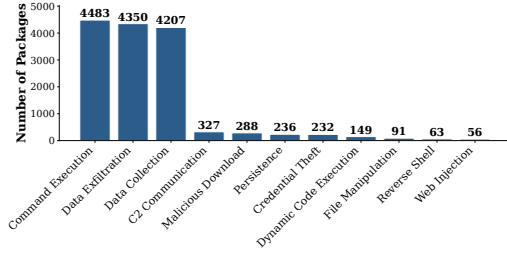


Figure 1: Behavior Distribution

consensus rate among experts, confirming the reliability of our extraction approach.

Behavior Summary Validation. To evaluate the accuracy of LLM-generated behavior summaries, we randomly sample 500 summaries and have three security experts independently verify whether each summary correctly describes the malicious behavior. A summary is marked as correct when at least two experts agree. The validation achieves 97.0% accuracy (485/500 correct) with 97.6% consensus rate and Fleiss’ Kappa of 0.74, indicating substantial inter-rater agreement. Among the 15 incorrect samples, 8 cases miss important behaviors that the code actually has, 4 cases claim behaviors that the code does not have, and 3 cases describe behaviors that do not match actual code behavior.

Taxonomy Analysis. Figure 1 shows the distribution of 11 malicious behavior categories across our dataset. The three most prevalent behaviors are command execution (4,483 packages), data exfiltration (4,350), and data collection (4,207), which together form the core attack pattern in NPM malware: collecting sensitive information and transmitting it to attacker-controlled servers via shell commands. Other notable behaviors include C2 communication (327), malicious download (288), and persistence (236), which indicate more sophisticated attack chains involving remote control and long-term access. Less frequent but critical behaviors such as credential theft (232), dynamic code execution (149), and reverse shell (63) reflect targeted attack objectives including runtime payload delivery and interactive remote access.

Malicious packages typically combine multiple behaviors, with a mean of 2.4 and a median of 3 per package. Across all pairs of the 11 categories, data collection and data exfiltration co-occur most frequently in 4,144 packages, followed by command execution with data exfiltration (3,020) and data collection (2,903), confirming a dominant *collect-and-exfiltrate* pattern in which shell commands serve as the shared mechanism for gathering and transmitting sensitive information. Rarer pairings such as credential theft with data exfiltration and command execution with persistence (158) point to the same long-term-access chains.

3.4 Detection Tools Selection

Tool identification. To collect candidate tools, we start from two established surveys of open-source supply-chain attacks, Backstabber’s Knife Collection [34] and the SoK of Ladisa et al. [24]. From these two papers we follow both their references and the papers that cite them, and we further search Google Scholar, the ACM Digital Library, and IEEE Xplore with terms such as *npm*, *malicious package*, *supply chain*, and *malware detection*, repeating this expansion until no new tool appears. The search yields 494 unique

papers. Among them, 79 concern the npm or JavaScript ecosystem, and 61 of those propose or apply a detector for malicious packages. The remaining papers study other ecosystems, measure the threat without building a detector, or address different problems such as vulnerability or hallucination detection.

We select tools from these candidates based on four criteria: (1) **Diversity**: covering static, dynamic, ML, and LLM approaches to ensure comprehensive coverage; (2) **Impact**: prioritizing tools published at top venues, highly cited, or developed and deployed by major vendors such as Microsoft and Datadog; (3) **Recency**: published or significantly updated within 2020–2026; (4) **Availability**: publicly accessible with API or CLI interfaces to ensure reproducibility.

We collected 11 NPM malicious package detection tools representing 16 distinct detection variants from academic research and industry practice. The evaluation includes: (1) **Static analysis tools**: GuardDog, OSSGadget, and GENIE, which analyze source code and metadata without execution; (2) **Dynamic analysis tool**: Packj, configured for both static analysis and dynamic trace modes; (3) **Machine learning tools**: SAP implemented with three classifiers (XGBoost, Random Forest, and Decision Tree), MalPacDetector implemented with three classifiers (MLP, Naive Bayes, and SVM), Cerebro which employs BERT to learn semantic patterns from behavior sequences, ProfMal which classifies behavior graphs built from combined static and dynamic analysis, and EMPHunter which clusters installation scripts and ranks outliers without requiring malicious examples. Although MalPacDetector uses LLMs, we classify it as ML-based as its features are ultimately constructed and classified by conventional ML models; (4) **LLM-based tools**: SocketAI, developed by Socket Inc., operating through a three-stage analysis pipeline with Initial Report generation, Critical Reports analysis, and Final Report synthesis, and IntelGuard, which retrieves known-malicious examples from a threat-intelligence knowledge base to ground the judgment of the LLM. To manage computational expenses, we configured SocketAI and IntelGuard with GPT-4.1 mini as the underlying language model, utilizing the Final Report output for malicious package classification. Table 1 summarizes the detection tools.

3.5 Experimental Setup

Environment. We evaluate detection tools on our unified dataset of 6,420 malicious package versions and 7,288 benign packages constructed as described in Section 3.1. Each tool processes packages using default configuration parameters to ensure reproducibility and reflect real-world deployment scenarios. Static analysis tools (GuardDog, GENIE, OSSGadget, Packj_static), dynamic analysis tool (Packj_trace), machine learning-based tools (SAP variants, MalPacDetector variants, Cerebro), and LLM-based tool (SocketAI) process packages directly through their respective detection pipelines. All experiments are conducted on Ubuntu 22.04.5 LTS systems equipped with 32GB RAM and Intel Xeon processors.

4 Empirical Results

4.1 RQ1: Detection Effectiveness

Results & Analysis. Table 4 presents the detection performance of all evaluated tools. Detection effectiveness varies dramatically

Table 3: Taxonomy of Malicious Behavior Categories

Category	Description	Category	Description
Command Execution	Executing OS-level shell commands via child process APIs (e.g., <code>exec</code> , <code>spawn</code>)	Data Exfiltration	Transmitting collected data to attacker-controlled servers (e.g., HTTPS POST, DNS queries)
Data Collection	Gathering host and environment information (e.g., <code>hostname</code> , <code>username</code> , <code>network interfaces</code>)	C2 Communication	Establishing persistent bidirectional connections to remote servers for receiving commands
Malicious Download	Fetching and executing remote payloads or platform-specific binaries	Persistence	Maintaining long-term access by injecting code into existing applications (e.g., <code>Discord modules</code>) or installing backdoors
Credential Theft	Stealing authentication materials (e.g., SSH keys, API tokens, browser cookies, crypto wallets)	Dynamic Code Exec.	Constructing and executing code at runtime via <code>eval</code> or <code>deobfuscation</code>
File Manipulation	Unauthorized file system operations such as writing payloads to disk or deleting evidence	Reverse Shell	Opening interactive shell connections to remote servers
Web Injection	Injecting malicious content into web pages (e.g., <code>iframe injection</code> , <code>form-jacking</code> , <code>phishing redirects</code>)		

Table 4: Performance Comparison of Detectors on Datasets

Type	Detector	Accuracy	Precision	Recall	F1-Score
Static-Based	OSSGadget	53.87%	50.41%	91.56%	65.02%
	GuardDog	93.97%	96.99%	89.92%	93.32%
	GENIE	74.72%	99.76%	46.14%	63.10%
	Packj_static	53.91%	50.41%	98.69%	66.73%
	Packj_trace	63.91%	56.82%	95.47%	71.24%
ML-Based	SAP_DT	83.86%	99.88%	65.62%	79.21%
	SAP_RF	84.16%	99.88%	66.26%	79.67%
	SAP_XGB	89.99%	94.47%	83.52%	88.66%
	MalPacDetector_MLP	84.90%	97.03%	73.00%	83.32%
	MalPacDetector_NB	85.15%	97.74%	72.92%	83.53%
	MalPacDetector_SVM	85.70%	98.00%	73.81%	84.20%
	Cerebro	81.52%	99.23%	58.80%	73.85%
	ProfMal	88.52%	97.55%	77.45%	86.34%
LLM-Based	SocketAI	73.58%	98.21%	44.41%	61.16%
	IntelGuard	96.32%	98.24%	93.82%	95.98%

Table 5: EMPHunter Ranking Performance on Malicious Versions Declaring Install Hooks

Metric	Top-1	Top-3	Top-5	Top-10
Recall	48.57%	74.89%	76.03%	80.46%
FPR	0.46%	2.03%	3.82%	8.28%
Precision	48.57%	24.96%	15.21%	8.05%
mAP@K	48.57%	61.53%	61.79%	62.35%

Note: Top-K considers the K highest-ranked candidates that EMPHunter reports, and a malicious version counts as recalled when it appears among them. mAP@K averages the reciprocal rank $1/r$ of each malicious version, where the term is zero when the version falls outside the top K.

across paradigms. Among static-based tools, GuardDog achieves the best overall balance with an F1-score of 93.32%, while GENIE sacrifices recall for near-perfect precision and Packj_static does the opposite, generating 6,234 false positives to recover 98.69% of malicious packages. Dynamic analysis improves recall to 95.47% but compounds the false positive problem, with Packj_trace producing 4,656 FP. ML-based tools exhibit a consistent precision-recall trade-off: SAP_DT and SAP_RF attain near-perfect precision yet miss roughly one-third of malicious packages, whereas SAP_XGB, the MalPacDetector variants, and ProfMal offer a more balanced trade-off with F1-scores ranging from 83.32% to 88.66%. The sharpest contrast appears within the LLM paradigm. SocketAI achieves only 44.41% recall, missing 3,569 packages, which renders it impractical as a standalone detection tool, whereas IntelGuard attains the highest F1-score of all evaluated tools (95.98%) by grounding its judgments in retrieved threat intelligence knowledge base. EMPHunter ranks candidate packages rather than classifying them and is therefore reported separately (Table 5).

We analyze the detection logic of each tool through source-code inspection to understand why these performance differences exist.

Capability-end tools flag any invocation of sensitive APIs regardless of context. Packj_static maps 56 JavaScript APIs to 10 permission categories and flags any matching package, achieving 98.69% recall but a signal-to-noise ratio of 1:71. OSSGadget’s false

positives arise largely from matching keywords such as `curl` and `wget` in README files and comments rather than executable code, a structural flaw no rule update can fully resolve.

Intent-end tools impose stricter evidence requirements at the cost of recall. GENIE’s `thet-os` query demands ≥ 3 unique OS data sources flowing to the same HTTP sink, creating a step-function boundary: multi-source exfiltration is caught with near-perfect precision, while single- or dual-source attacks are missed entirely, and non-HTTP exfiltration channels are excluded by definition. SAP_DT/RF achieve 99.88% precision because their learned boundaries are so narrow that only patterns closely matching the 102-sample training set are flagged.

GuardDog sits between these two extremes by using taint analysis that requires end-to-end data flow from sensitive sources to network sinks as partial evidence of malicious intent, combined with an allowlist of known legitimate hook patterns to suppress false positives. Notably, just two of its 22 rules account for over 90% of detections: `npm-install-script` targeting lifecycle script abuse and `shady-links` targeting suspicious outbound URLs, revealing that NPM malware is homogeneous in its attack entry point.

Among ML-based tools, feature quality matters more than classifier choice. MalPacDetector’s 24 security-oriented binary features yield consistent accuracy across all three classifiers, while SAP’s 140 features are structurally compromised: 65% are file-extension counts with no security semantics, and its two ostensibly discriminative features turn out to be reverse signals, with dangerous token counts and Base64 chunk counts both scoring higher in benign packages than in malicious ones. ProfMal confirms the same principle from the opposite direction. Its behavior graphs, built by object-sensitive static analysis and completed through dynamic execution, are the richest feature representation in this group and yield its highest precision (97.55%). Yet even graph evidence trades recall away (77.45%, 1,448 missed versions), because attacks confined to install scripts leave little for a graph to capture.

LLM-Based Tools. SocketAI’s low recall has three structural causes: (1) its Critical Reports stage explicitly instructs the LLM to challenge prior judgments about malicious behavior, systematically downgrading initially correct detections; (2) LLMs produce conservative scores, placing genuine malware below the 0.5 classification threshold; and (3) a 20-file-per-package limit causes payloads in deep directory structures to be skipped entirely. At 146 seconds per package, it is also impractical for large-scale deployment. IntelGuard avoids these failure modes by design. It slices package files, retrieves similar known-malicious examples from a knowledge base distilled from over 8,000 threat intelligence reports, and lets the LLM

judge each slice against these precedents. Anchoring the judgment to retrieved evidence, rather than asking the model to challenge itself, preserves 93.82% recall at 98.24% precision (108 false positives). The two tools even run the same underlying model, GPT-4.1 mini, so the 49 point recall gap between them comes entirely from how each pipeline structures the model’s evidence.

Runtime Performance. We evaluate runtime cost on selected packages averaging 47.4 files and 2,961.8 lines of code. GuardDog processes packages fastest at 2.55 seconds due to efficient Semgrep execution. OSSGadget requires 7.94 seconds for 133 pattern rules. SAP_XGB needs 9.58 seconds for feature extraction and inference. Packj_trace demands 17.90 seconds for dynamic analysis. SocketAI is slowest at 146.34 seconds due to multiple LLM queries. For large-scale scanning, GuardDog processes 1,412 packages per hour while SocketAI processes only 24.

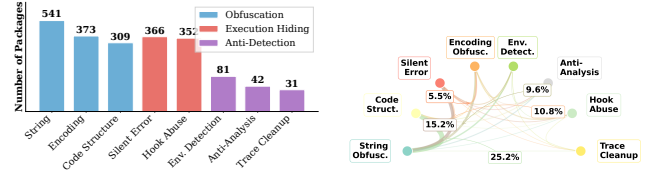
Ranking-based detection. EMPHunter clusters the installation scripts of newly uploaded packages and surfaces outliers as candidate malware, so we evaluate it as a ranking task. Each ranking instance mixes one malicious version with 111 benign packages, and we record the rank at which the malicious version appears. Reading installation scripts only, the tool covers the 5,236 malicious versions that declare an install hook (81.6% of the corpus) and leaves the remaining 1,184 outside its design. Within this scope it recalls 80.46% at Top-10 with an 8.28% false-positive rate (mAP@10 of 62.35%, Table 5). Top-K precision is capped by construction. With one malicious version per instance, at most one of the K inspected candidates can be a true positive, so precision cannot exceed $1/K$, and the 8.05% at Top-10 sits close to that 10% ceiling. EMPHunter thus examines the spot that code-analysis tools ignore, the install commands inside package.json, which makes it a complement to those tools rather than a standalone detector.

Finding 1: Malicious and benign packages invoke the same OS APIs, so every detector must separate *capability* (what code can do) from *intent* (what code aims to do). Tools that flag every sensitive API catch most malware but drown in false positives, while tools that demand strict evidence stay precise but miss attacks. GuardDog strikes the best balance among conventional tools (93.32% F1) through end-to-end taint analysis, and IntelGuard goes further (95.98% F1) by grounding LLM reasoning in examples retrieved from its knowledge base.

4.2 RQ2: Fine-grained Behavioral Analysis

While RQ1 evaluated overall detection effectiveness, RQ2 examines detection capabilities across specific malicious behaviors and evasion techniques.

4.2.1 RQ2.1: Behavioral Detection Capability. Results & Analysis. Detection performance varies significantly across the 11 behavior categories and 15 tool variants, as shown in Figure 3. Packj_static achieves the broadest coverage, exceeding 80% for 10 of 11 categories through comprehensive API monitoring. In contrast, GENIE, SAP_DT/RF, and ProfMal show critical gaps in dynamic code execution, C2 communication, and web injection, reflecting the limitations discussed in RQ1. GuardDog exhibits the most uneven profile among well-performing tools: it exceeds 94% for data exfiltration and command execution but drops to only 7% for dynamic code



(a) Evasion Distribution

(b) Evasion Co-occurrence

Figure 2: Evasion Technique Distribution

execution and 27% for web injection, because its Semgrep rules were not designed to cover these categories.

High-detectability behaviors. Data collection, data exfiltration, and command execution achieve mean detection rates above 75% across all tools. Beyond their prevalence, these behaviors benefit from a coupling effect: they rarely occur in isolation, and co-occurrence amplifies detection signals. When data collection and exfiltration co-occur, mean detection rises from 40.0% and 72.2% individually to 85.4%. The effect is most pronounced for SAP_DT, which jumps from 3.2% to 79.3%, because a single `os.hostname()` call is ambiguous, but the chain `os.hostname()` → `JSON.stringify()` → `https.request()` constitutes unambiguous intent evidence.

Low-detectability behaviors. Dynamic code execution, web injection, and credential theft show the widest cross-tool variation and the lowest mean detection rates. These behaviors do not form behavioral chains, so there is no coupling effect to amplify the signal. More critically, web injection targets browser-side DOM manipulation, but all evaluated tools analyze server-side Node.js code, creating a structural attack surface mismatch that no detection rule can bridge. Even IntelGuard, the strongest detector overall, drops from 98.5% on data exfiltration to 62.5% on web injection, which places the limit on the analysis surface rather than on the sophistication of the reasoning applied to the code. Credential theft faces a similar problem: the APIs involved are identical to those used by legitimate authentication libraries, leaving feature-based tools with no discriminative signal.

Finding 2: Detection difficulty is shaped by two structural factors. Behavioral coupling amplifies intent signals when behaviors co-occur: the collection+exfiltration chain raises SAP_DT detection from 3.2% to 79.3%. Attack surface misalignment makes certain behaviors structurally undetectable: web injection targets browser-side DOM while tools analyze Node.js code, and credential theft uses APIs identical to legitimate authentication libraries.

4.2.2 RQ2.2: Evasion Technique Analysis. To measure how well each detector copes with evasion, we first label which technique each malicious package uses. We adopt eight evasion categories established in prior studies of malicious packages [24, 34], from obfuscation to trace cleanup. For each category we assemble a set of pattern rules, adapting signatures from prior work and public tools and adding our own for the cases they miss, 53 rules in total, and run them over each package’s malicious code and package.json scripts. An expert then reviews every match, and a package is labeled evasion-free only when none survives review.

Figure 2a shows the distribution of 8 evasion technique categories, and Figure 4 presents detection rates against each technique.

Results & Analysis. Among 6,420 malicious packages, 80.3% employ no evasion technique at all. Among the 19.7% that do, string obfuscation is most prevalent (541 packages), followed by encoding obfuscation, silent error handling, and hook abuse. Despite their rarity, environment detection and anti-analysis are the hardest to detect, with mean detection rates of 39.5% and 52.7% respectively.

Obfuscation techniques. String and code structure obfuscation defeat feature-level tools by altering the surface form from which features are computed. Attackers rename identifiers with hex-encoded strings such as `_0x27b98b` and flatten control flow, stripping the statistical signals that SAP_DT/RF and Cerebro rely on, all three falling to 26.6% or below on string obfuscation. Packj_static holds at 86.0% because it monitors API calls regardless of how identifiers are named.

Anti-analysis techniques. Anti-analysis exhibits the largest detection gap of all categories, from 5% for SAP_DT/RF and Cerebro to 100% for Packj_static. Attackers use self-modifying *eval* wrappers with integrity checks: if a scanner modifies the payload, the hash changes and the code refuses to execute, defeating any tool that instruments or rewrites code during analysis. Packj_static bypasses this entirely by monitoring system calls at the OS level, never touching the code itself. Trace cleanup shows an equally sharp split: SAP_DT/RF and GENIE detect below 10% because evidence is deleted before feature extraction completes, while GuardDog and MalPac-NB both reach 97% through pattern matching on file deletion APIs.

Paradigm-level pattern. Evasion effectiveness depends on whether the technique targets the same abstraction level as the detector. Obfuscation defeats feature-level tools but leaves API-level tools unaffected. Anti-analysis and environment detection defeat code-level tools by using APIs identical to legitimate system queries, but fail against syscall-level tools that observe actual OS interactions rather than inspecting code. A richer representation does not exempt a tool from this rule. ProfMal builds behavior graphs from static and dynamic analysis, yet falls to 11.9% on anti-analysis and 25.9% on environment detection, tracking SAP and Cerebro rather than the syscall-level tools, because those graphs still derive from the code surface these techniques alter. What matters is where a tool’s decision boundary is written. IntelGuard never falls below 66.7% on any technique and averages 81.3%, within a tenth of a point of OSSGadget’s 81.2%. A regular expression and a retrieved precedent both live outside the package, so rewriting its surface moves neither. No single technique defeats all abstraction levels.

Evasion combinations. Attackers frequently layer techniques, with code structure combined with string obfuscation accounting for 36.81% of all combinations. Packages using 2–3 techniques are detected by fewer tools on average than those using only one, but packages using 4 or more are detected by more tools than any other group, because heavy obfuscation produces its own anomalies: elevated entropy, abnormal code structure, and large decoding scaffolds. Evasion code intended to hide intent ends up signaling that something is being hidden.

Finding 3: Most attackers do not attempt evasion at all, as npm performs no scanning before publication and simple attacks remain effective. Among those that do, adding techniques offers

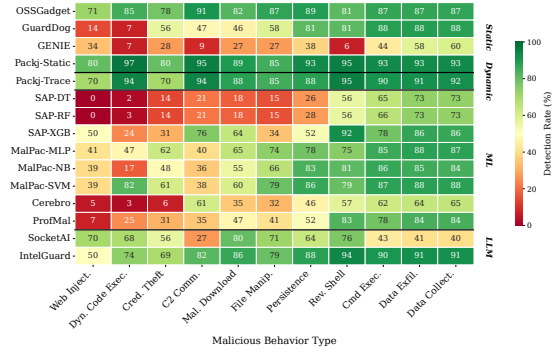


Figure 3: Detection Rates Across Malicious Behaviors

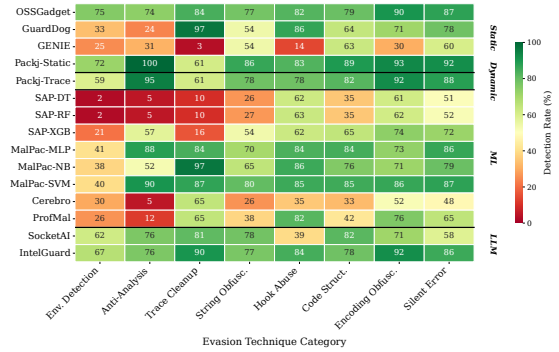


Figure 4: Detection Rates Against Evasion Techniques

little protection. Packages combining four or more are caught by more tools than those using two or three, since heavy obfuscation leaves traces of its own. Attackers respond by abandoning failed techniques rather than refining them, moving instead to layers that tools do not inspect.

4.2.3 RQ2.3: Installation Script Abuse. Results & Analysis. Among 6,420 malicious packages, 4,636 (72.21%) exploit installation scripts as attack vectors, counting a package only when the code confirmed malicious in its package. `json` is itself a declared hook, so a benign build tool that merely declares one never enters the count. Preinstall scripts dominate, appearing in 61.15% of all malicious packages, followed by postinstall (6.74%) and install scripts (2.20%). Malicious code concentrates in two locations: `index.js` in 46.9% of packages and the `package.json` scripts section in 38.2%. More strikingly, 1,362 packages (21.2%) rely entirely on install scripts with no malicious JavaScript code in any file.

Preinstall scripts are the preferred entry point for a structural reason: they execute automatically during `npm install` with full user privileges before dependency resolution, before security tooling can activate, and regardless of whether the installation ultimately succeeds. The silent execution pattern `"preinstall": "node index.js > /dev/null 2>&1"` further ensures no console output reveals the attack. This is not a vulnerability being exploited but a design decision: npm’s lifecycle hook mechanism was built on the assumption that publishers are benign, while its open publication model provides no such guarantee.

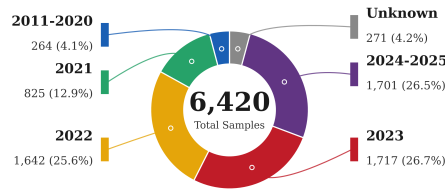


Figure 5: Package Temporal Distribution

Table 6: Evasion Techniques Across Time Periods

Evasion Technique	Time Period					Total
	≤2020 (249)	2021 (817)	2022 (1,596)	2023 (1,704)	2024-25 (1,622)	
String Obfuscation	31 (12.4%)	77 (9.4%)	149 (9.3%)	104 (6.1%)	150 (9.2%)	541
Encoding Obfuscation	48 (19.3%)	136 (16.6%)	47 (2.9%)	90 (5.3%)	37 (2.3%)	373
Code Structure Obfusc.	10 (4.0%)	63 (7.7%)	102 (6.4%)	—	102 (6.3%)	309
Silent Error Handling	—	59 (7.2%)	112 (7.0%)	63 (3.7%)	106 (6.5%)	366
Hook Abuse	12 (4.8%)	55 (6.7%)	59 (3.7%)	49 (2.9%)	152 (9.4%)	352
Environment Detection	29 (11.6%)	—	—	—	—	81
Anti-Analysis	—	—	—	21 (1.2%)	—	42
Trace Cleanup	—	—	—	—	—	31

Note: Only top-5 evasion techniques per period are shown. “—” indicates the technique was not among the top five for that period.

The fundamental detection difficulty is not technical but semantic. Legitimate build tools such as *husky*, *patch-package*, and *prisma generate* use the exact same preinstall and postinstall hooks for entirely benign purposes. A tool that flags all install scripts produces massive false positives as Packj does with 6,243 FP; a tool that requires stronger evidence of malicious intent misses most attacks as GENIE does with only 14.77% hook abuse detection. The 21.2% of install-only malware sharpens this further: when the entire attack is a single command string in package.json invoking a separate JavaScript file, there is no obfuscation or suspicious API to detect, just a pattern indistinguishable from any legitimate build script.

Finding 4: 72.21% of malicious packages exploit install scripts because npm lifecycle hooks guarantee execution without sandboxing. The 21.2% that embed attacks entirely in package.json expose an architectural blind spot: every code-analysis tool assumes malicious behavior lives in code, and when it does not, no rule refinement helps.

4.3 RQ3: Temporal Evolution Analysis

While RQ1 and RQ2 evaluated detection effectiveness on our complete dataset, RQ3 examines whether detection tools maintain consistent performance as attack techniques evolve over time.

Timestamp Collection. We collect publication timestamps via the NPM Registry API, retrieving timestamps for 6,149 of 6,420 packages (98.5%). As shown in Figure 5, 95.7% of packages were published between 2021 and 2025, reflecting the growing threat of supply chain attacks and increased detection efforts in recent years.

4.3.1 Detection Performance Evolution. Figure 6 presents detection rate trends across time periods for all 15 tool variants.

Results & Analysis. Detection tools exhibit divergent temporal patterns. Packj_static maintains above 97% across all periods with only $\pm 1.15\%$ variance, and IntelGuard stays above 91% from 2021 onward, while SAP_DT drops from 87.15% in 2021 to 39.49% in 2023, ProfMal falls from 90.42% to 65.93% over the same years, Cerebro

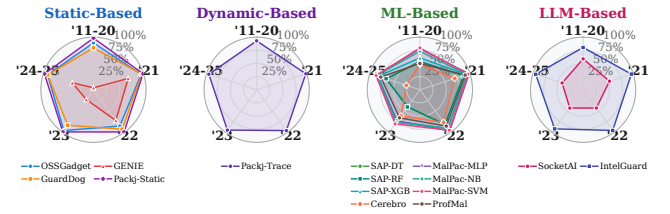


Figure 6: Detection Rate Comparison Across Time Periods by Tool Category

collapses from 76.61% to 27.04% between 2022 and 2024–2025, and GENIE oscillates between 3.79% and 72.17%. All tools use fixed detection parameters, so the divergence reflects changes in the malware population, not tool updates.

The ML degradation contradicts the conventional concept drift explanation. Between 2022 and 2023, obfuscation decreased and malware became simpler, not more sophisticated. SAP_DT collapsed because minimal-footprint malware became statistically indistinguishable from benign packages in feature space, a phenomenon we term *concept convergence*. ProfMal, published four years later and trained on a more recent corpus, traces the same trough from 90.42% to 65.93%, which shows that convergence is a property of the feature space rather than a defect of any one model. Its partial recovery in 2024–2025 was not adaptation: the hook-abuse surge happened to trigger SAP’s install-script feature, shifting the malware population back into its coverage. Cerebro’s collapse follows a different mechanism: its BERT embeddings peak on 2022 malware because its training corpus aligns with that era, then fail silently as attack code evolves beyond its learned representations. GENIE’s 68-point oscillation reflects a third failure mode: not degradation but query-pattern misalignment, peaking when dominant attacks match its CodeQL conditions and collapsing when they do not. GuardDog is the only tool showing genuine recovery, demonstrating that pattern-based tools can adapt through rule maintenance in a way that ML models fundamentally cannot. IntelGuard remains stable without any rule maintenance because it grounds each judgment in retrieved precedent rather than in learned surface statistics, so shifts in code appearance leave its evidence base intact. Underlying all these patterns is a single principle: tools that detect what code *does* are temporally robust because attack objectives are invariant, while tools that detect code surface properties are fragile because appearances evolve freely.

4.3.2 Evasion Technique Evolution. Table 6 presents the evolution of evasion techniques across 8 categories from 2011 to 2025.

Results. Evasion use grows from 94 packages in 2011–2020 to 358 in 2024–2025, a 3.8x increase, but the corpus grows 6.5x over the same span (249 to 1,622), so the share of malware carrying any evasion falls from 37.8% to 22.1%. Encoding obfuscation dominated early malware but declined from 19.3% to 2.3%. String obfuscation stays between 6.1% and 12.4% across all periods. Hook abuse rebounds from 2.9% in 2023 to 9.4% in 2024–2025, the most prevalent technique in that period. Anti-analysis stays rare throughout.

Analysis. Encoding obfuscation gives way to hook abuse across the full span, 19.3% $\rightarrow$ 2.3% for the former and 4.8% $\rightarrow$ 9.4% for the latter. Hook abuse exploits npm’s trusted execution model, where

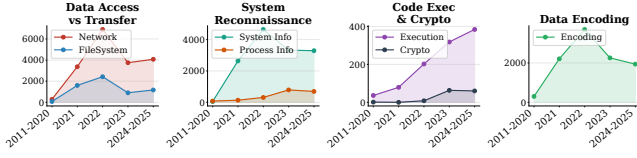


Figure 7: Evolution of Sensitive API Usage in Malicious NPM Packages Over Time

preinstall/postinstall scripts run with full privileges, bypassing code-level analysis entirely. The decline of encoding obfuscation explains the ML collapse. SAP models trained on 2021 data learned high-entropy patterns as malicious indicators, and once attackers moved to clean code with standard APIs, those features stopped separating malicious from benign, causing the 47.66 percentage point drop in 2023. ProfMal follows the same path despite a more recent training corpus, so the shift defeats the feature space rather than one model. GuardDog recovers to 91.42% because its Semgrep rules match manifest-level hook patterns that other tools miss, and IntelGuard holds above 91% because retrieved precedent does not age as code appearance changes.

Finding 5: Temporal robustness follows the origin of a detector’s decision boundary, not its paradigm. Every boundary fitted to a corpus ages, and ProfMal proves that representation richness is no defense, since its behavior graph troughs as steeply as SAP’s statistics, while IntelGuard stays above 91% by grounding verdicts in retrieved evidence. SAP and Cerebro age by different routes, through convergence toward benign code and through stale embeddings, and GuardDog alone recovered, through rule updates rather than retraining.

4.3.3 Sensitive API Evolution. To understand how malicious code patterns evolve beyond evasion techniques, we analyze the sensitive APIs invoked by malicious packages across time periods.

API Extraction. We extract sensitive API calls from malicious code snippets using *tree-sitter* [43], a parsing library that constructs abstract syntax trees (ASTs) for JavaScript code. For each code snippet, we parse the AST and identify API invocations through three mechanisms: (1) call expressions matching known sensitive functions, (2) member expressions accessing module methods (e.g., *fs.readFile*, *os.hostname*), and (3) string literals containing API names to capture obfuscated dynamic calls. Following the sensitive API taxonomy established in prior work [9, 52], we categorize extracted APIs into seven categories: network, filesystem, system_info, process_info, execution, encoding, and crypto. This extraction successfully processes 6,239 packages, identifying 53,576 API invocations.

Results & Analysis. Figure 7 presents the temporal evolution of sensitive API usage across 6,239 packages. Most categories peaked in 2022 and declined thereafter, while execution APIs grew continuously across all periods. The average API count per package declined from 12.52 in 2021 to 6.82 in 2023, and by 2024–2025, 12.6% of malware invokes only 1–2 APIs total, up from 5.1% in 2021, with 51% of these minimal packages importing only *child_process* to execute a single shell command. Crypto APIs emerged after 2022 and have grown steadily, suggesting increasing use of encrypted communication channels. This reflects a structural shift from 2021-era full reconnaissance attacks, which collected hostname, environment

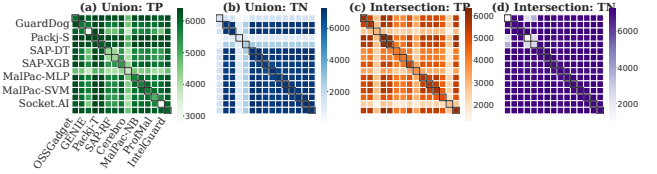


Figure 8: Complementary Analysis of Detection Tools: TP/TN Performance Matrix

Table 7: Top Tool Combinations and Their Gain over the Stronger Single Tool

Union Strategy					Intersection Strategy				
Combination	Prec.	Rec.	F1	Δ	Combination	Prec.	Rec.	F1	Δ
IntelGuard + MalPac_SVM	96.81	97.24	97.02	+1.05	Packj_static + IntelGuard	98.48	92.66	95.48	-0.49
GuardDog + IntelGuard	96.04	97.41	96.72	+0.74	Packj_trace + IntelGuard	98.72	90.23	94.29	-1.69
IntelGuard + MalPac_NB	96.65	96.74	96.70	+0.72	GuardDog + Packj_static	97.39	89.07	93.04	-0.28
SAP_RF + IntelGuard	98.21	94.89	96.52	+0.55	GuardDog + IntelGuard	99.48	86.32	92.44	-3.54
GuardDog + SocketAI	96.45	95.14	95.79	+2.47	Packj_static + MalPac_SVM	98.23	87.27	92.43	+8.23

Note: Δ is the F1 gain over the stronger of the two tools used alone. Union flags a package when either tool detects it; Intersection requires both. In each column, the first four rows are the top pairs by F1, and the row below the rule is the strongest IntelGuard-free pair not among them, shown for reference.

variables, and user information before exfiltrating via *https.request*, toward single-step execution: when the network operation is embedded inside a shell command such as `exec('curl @tacker.com | sh')`, API-level tools tracking *https.request* as a network sink cannot observe it at all, and the behavioral coupling effect that amplifies detection signals disappears entirely.

Finding 6: Sensitive API usage has shifted from broad multi-category reconnaissance toward minimal execution-focused attacks. Single-step shell command attacks eliminate the behavioral coupling that amplifies detection signals and bypass API-level sink definitions entirely, leaving process-execution-level monitoring as the only reliable detection surface against this pattern.

4.4 RQ4: Tool Complementarity Analysis

While RQ1-RQ3 revealed significant performance variations among individual tools, RQ4 examines whether strategic tool combinations can overcome these limitations. We evaluate pairwise combinations using two strategies: union (flagging packages detected by either tool) and intersection (requiring both tools to agree).

Results & Analysis. Table 7 and Figure 8 present the top combinations across all 15 tool variants. For union, IntelGuard + MalPac_SVM achieves the best F1 at 97.02% with 97.24% recall, above any individual tool. For intersection, Packj_static + IntelGuard leads with 95.48% F1 and 98.48% precision. Cross-methodology pairs consistently outperform same-methodology ones, yet paradigm diversity is a proxy for complementarity rather than its cause.

Two opposing forces govern combination effectiveness, detection complementarity and false-positive introduction. We quantify complementarity with McNemar’s χ^2 . GuardDog and SocketAI reach $\chi^2 = 1,920$, while SAP_DT and SAP_RF reach only 39, because both extract identical features and disagree on almost nothing. High χ^2 is necessary but not sufficient. GuardDog and Packj_static reach $\chi^2 = 4,020$ yet fail in union, because Packj_static adds only 665 true positives while introducing 6,015 false positives, collapsing combined precision from 97% to 50%. What separates a useful partner

from a useless one is therefore the detection mechanism, not the paradigm label. The intra-ML pair Cerebro + MalPac_SVM succeeds because Cerebro reads BERT embeddings while MalPac_SVM reads 24 binary AST features, so the packages each tool misses are largely different. SAP_DT + SAP_RF pairs two different classifiers over the same 140 features, misses almost identical packages, and gains nothing from combination. The same accounting explains a counterintuitive result. IntelGuard and GuardDog, the two strongest individual tools, are on average hurt by union at -0.05 and -0.04 F1, because most partners introduce more false positives than additional true positives, while SocketAI benefits most at $+0.08$ F1.

For intersection, Packj_static + IntelGuard succeeds where union fails because requiring both tools to agree filters out each tool's independent false positives, IntelGuard grounds its judgment in retrieved precedent while Packj_static flags legitimate system calls, but packages flagged by both are almost always genuine malware.

Finding 7: IntelGuard + MalPac_SVM achieves the best union F1 (97.02%), and the gain over the stronger tool alone shrinks as that tool grows stronger, from $+2.47$ for the best IntelGuard-free pair down to $+1.05$. The principle is asymmetric: the strongest tool degrades because partners add more false positives than new true positives, while the weakest tool benefits most because a strong partner fills its blind spots at low cost.

5 Implications

For Practitioners. Tool selection is a budget decision rather than a single recommendation. Where an LLM budget exists, IntelGuard + MalPac_SVM union is the most accurate configuration we measured at 97.02% F1, and a tiered deployment keeps its cost bounded, since a cheap pass such as GuardDog screens every package at 2.55 s each while the LLM verifies only the flagged minority. Where no LLM budget exists, Cerebro + MalPac_SVM union reaches 95.15% F1 entirely offline with no API dependency. Pipelines that cannot tolerate false positives should prefer intersection, where GuardDog + Packj_static reaches 93.04% F1 at 97.39% precision using two static tools. As hook abuse surged to 9.4% in 2024–2025, organizations should consider disabling script execution via `npm install --ignore-scripts` for untrusted packages. Teams unwilling to retrain ML-based tools quarterly should switch to behavioral detection tools, which remain stable over time.

For Researchers. The capability-intent gap calls for mechanisms that supply partial intent evidence without the strict formal verification that limits recall. Taint analysis and behavior-chain detection resolve the ambiguity through data-flow evidence rather than surface-level features. Retrieved precedent is a third such mechanism, and it is knowledge-driven rather than data-driven. IntelGuard pairs malicious code with the expert reasoning for why that behavior is malicious, so the model judges semantics against a prior intent judgment instead of matching syntax or a boundary learned from a corpus, and RQ3 finds such a judgment does not age. Reasoning is no substitute for that knowledge, since SocketAI's agentic loop of three self-critical rounds costs $57\times$ GuardDog's latency and still loses 49 recall points to one grounded call. Dynamic code execution, web injection, and credential theft remain structurally underdetected, the most pressing open problems.

6 Threats to Validity

Internal Validity. Ground truth depends on expert annotation; three experts achieved 97.4% consensus, but edge cases may be mislabeled. Tools use default parameters for reproducibility; different settings could affect results. LLM-based extraction achieves 97.2% accuracy on 500 validated samples, and the remaining 2.8% errors may affect behavioral analysis.

External Validity. The dataset concentrates on packages from 2021–2025 (95.7%), for which per-period samples exceed 800 and the temporal findings are correspondingly robust. Finally, our benign set consists of highly downloaded packages. We chose them because their benign labels are the most reliable, since malware in a widely used package would be discovered quickly, and because they trigger the highest false-positive rates, which makes them the most demanding test of precision. Precision on the long tail of rarely downloaded packages, which we do not cover, may therefore differ.

Construct Validity. The behavioral taxonomy derives from LLM summaries and clustering. Alternative clustering parameters or different embedding models could produce different category boundaries. Evasion technique categorization involves subjective judgment. Some techniques overlap (e.g., encoding obfuscation and string obfuscation), and boundary definitions may vary across researchers, potentially introducing selection bias.

7 Conclusion

This paper evaluates 11 NPM detection tools with 16 variants on a unified dataset of 6,420 malicious and 7,288 benign packages. IntelGuard achieves the best F1 at 95.98%, GuardDog the best balance among conventional tools at 93.32%, and strategic combinations reach up to 97.21% accuracy. ML-based tools degrade severely because malware converged toward benign code as obfuscation became unnecessary in an unscanned ecosystem, not because attacks grew more sophisticated. Source-code inspection traces precision-recall trade-offs, temporal fragility, and combination effectiveness to one root cause, the ambiguity between code capability and malicious intent. Tools anchored to attack objectives stay stable while tools anchored to attack implementations do not, so detection should be designed around what malware must do.

8 Data Availability

Our dataset and evaluation framework are available online: <https://doi.org/10.6084/m9.figshare.31869370>.

Acknowledgments

This research is supported by the National Research Foundation, Singapore, and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG4-GC-2023-008-1B); by the National Research Foundation Singapore and the Cyber Security Agency under the National Cybersecurity R&D Programme (NCRP25-P04-TAICeN); and by the Prime Minister's Office, Singapore under the Campus for Research Excellence and Technological Enterprise (CREATE) Programme. Any opinions, findings and conclusions, or recommendations expressed in these materials are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, Cyber Security Agency of Singapore, Singapore.

References

- [1] 2025. NPMJS. <https://www.npmjs.com/>. Accessed: 2025-07-08.
- [2] 2026. Code and Dataset. <https://doi.org/10.6084/m9.figshare.31869370>. Accessed: 2026-03-27.
- [3] Rabe Abdalkareem, Vinicius Oda, Suhaib Mujahid, and Emad Shihab. 2020. On the impact of using trivial packages: An empirical case study on npm and pypi. *Empirical Software Engineering* 25, 2 (2020), 1168–1204. doi:10.1007/s10664-019-09792-9
- [4] Broadcom. 2021. Malicious coa and rc npm Packages Discovered. <https://www.broadcom.com/support/security-center/protection-bulletin/malicious-coa-and-rc-npm-packages-discovered>. Accessed: 2025-07-08.
- [5] Philipp Burckhardt. 2024. npm in Review: A 2023 Retrospective on Growth, Security, and Quirky Facts. <https://socket.dev/blog/2023-npm-retrospective>. Accessed: 2025-07-08.
- [6] Yulu Cao, Lin Chen, Wanwangying Ma, Yanhui Li, Yuming Zhou, and Linzhang Wang. 2023. Towards Better Dependency Management: A First Look at Dependency Smells in Python Projects. *IEEE Trans. Softw. Eng.* 49, 4 (April 2023), 1741–1765. doi:10.1109/TSE.2022.3191353
- [7] CISA. [n.d.]. Malware Discovered in Popular npm Package ua-parser-js. <https://www.cisa.gov/news-events/alerts/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js>. Accessed: 2025-07-08.
- [8] DataDog. 2022. GuardDog: A Threat Detection Tool. <https://github.com/DataDog/guarddog>. Accessed: 2025-07-08.
- [9] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Network and Distributed Systems Security (NDSS) Symposium 2021*. doi:10.14722/ndss.2021.23055
- [10] ESLint Team. 2018. Postmortem for Malicious Packages Published on July 12th, 2018. <https://eslint.org/blog/2018/07/postmortem-for-malicious-package-publishes/>. Accessed: 2025-07-08.
- [11] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing Malicious Package Updates in npm with a Lightweight Permission System. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 1334–1346. doi:10.1109/ICSE43902.2021.00121
- [12] Fabian Niklas Froh, Matias Federico Gobbi, and Johannes Kinder. 2023. Differential Static Analysis for Detecting Malicious Updates to Open Source Packages. In *Proceedings of the 2023 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (Copenhagen, Denmark) (SCORED '23)*. Association for Computing Machinery, New York, NY, USA, 41–49. doi:10.1145/3605770.3625211
- [13] Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting suspicious package updates. In *Proceedings of the 41st International Conference on Software Engineering: New Ideas and Emerging Results (Montreal, Quebec, Canada) (ICSE-NIER '19)*. IEEE Press, 13–16. doi:10.1109/ICSE-NIER.2019.00012
- [14] Matias F. Gobbi and Johannes Kinder. 2024. GENIE: Guarding the npm Ecosystem with Semantic Malware Detection. In *2024 IEEE Secure Development Conference (SecDev)*. 117–128. doi:10.1109/SecDev61143.2024.00017
- [15] Wenbo Guo, Shiwen Song, Jiaxun Guo, Zhengzi Xu, Chengwei Liu, Haoran Ou, Mengmeng Ge, and Yang Liu. 2026. Bridging Expert Reasoning and LLM Detection: A Knowledge-Driven Framework for Malicious Packages. In *Proceedings of the ACM Web Conference 2026 (United Arab Emirates) (WWW '26)*. Association for Computing Machinery, New York, NY, USA, 3554–3565. doi:10.1145/3774904.3792083
- [16] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. 2024. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (Echternach, Luxembourg) (ASE '23)*. IEEE Press, 166–177. doi:10.1109/ASE56229.2023.00135
- [17] Sajal Halder, Michael Bewong, Arash Mahboubi, Yinhao Jiang, Md Rafiqul Islam, Md Zahid Islam, Ryan HL Ip, Muhammad Ejaz Ahmed, Gowri Sankar Ramachandran, and Muhammad Ali Babar. 2024. Malicious Package Detection using Metadata Information. In *Proceedings of the ACM Web Conference 2024 (Singapore, Singapore) (WWW '24)*. Association for Computing Machinery, New York, NY, USA, 1779–1789. doi:10.1145/3589334.3645543
- [18] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: malicious NPM packages detector using behavior sequence knowledge mapping. In *Proceedings of the 33rd USENIX Conference on Security Symposium (Philadelphia, PA, USA) (SEC '24)*. USENIX Association, USA, Article 211, 18 pages.
- [19] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 1146–1158. doi:10.1145/3691620.3695492
- [20] Yiheng Huang, Wen Zheng, Susheng Wu, Bihuan Chen, You Lu, Zhuotong Zhou, Yiheng Cao, Xiaoyu Li, and Xin Peng. 2025. ProfMal: Detecting Malicious NPM Packages by the Synergy between Static and Dynamic Analysis. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 419–431. doi:10.1109/ASE63991.2025.00042
- [21] Intrinsic. 2018. Compromised npm Package event-stream. <https://medium.com/intrinsic-blog/compromised-npm-package-event-stream-d47d08605502>. Accessed: 2025-07-08.
- [22] RPTU Kaiserslautern-Landau. 2025. Remote Access Trojaner in npm Paket. <https://rptu.de/en/informationssicherheit/sicherheitswarnungen/details/news/remote-access-trojaner-in-npm-paket>. Accessed: 2025-07-08.
- [23] Datadog Security Labs. 2023. Open-Source Dataset of Malicious Software Packages. <https://github.com/DataDog/malicious-software-packages-dataset/tree/main/samples/npm>. Accessed: 2025-07-08.
- [24] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1509–1526. doi:10.1109/SP46215.2023.10179304
- [25] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference (Austin, TX, USA) (ACSAC '23)*. Association for Computing Machinery, New York, NY, USA, 71–82. doi:10.1145/3627106.3627138
- [26] Jasmine Latendresse, Suhaib Mujahid, Diego Elias Costa, and Emad Shihab. 2023. Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (Rochester, MI, USA) (ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 73, 12 pages. doi:10.1145/3551349.3556896
- [27] Robert Lemos. 2022. One-Third of Popular PyPI Packages Mistakenly Flagged as Malicious. <https://www.darkreading.com/application-security/one-third-pypi-packages-mistakenly-flagged-malicious>. Accessed: 2024-12-27.
- [28] Ningke Li, Shenao Wang, Mingxi Feng, Kailong Wang, Meizhen Wang, and Haoyu Wang. 2024. MalWuKong: Towards Fast, Accurate, and Multilingual Detection of Malicious Code Poisoning in OSS Supply Chains. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (Echternach, Luxembourg) (ASE '23)*. IEEE Press, 1993–2005. doi:10.1109/ASE56229.2023.00073
- [29] Wentao Liang, Xiang Ling, Chen Zhao, Jingzheng Wu, Tianyue Luo, and Yanjun Wu. 2026. Detecting Malicious Packages in PyPI and NPM by Clustering Installation Scripts. *IEEE Transactions on Software Engineering* 52, 1 (2026), 36–53. doi:10.1109/TSE.2025.3618952
- [30] S. Lloyd. 1982. Least squares quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137. doi:10.1109/TIT.1982.1056489
- [31] Microsoft. 2020. OSSGadget: Collection of Tools for Analyzing Open Source Packages. <https://github.com/microsoft/OSSGadget>. Accessed: 2025-07-08.
- [32] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. 2023. Beyond typosquatting: an in-depth look at package confusion. In *Proceedings of the 32nd USENIX Conference on Security Symposium (Anaheim, CA, USA) (SEC '23)*. USENIX Association, USA, Article 193, 18 pages.
- [33] Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the Feasibility of Supervised Machine Learning for the Detection of Malicious Software Packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security (Vienna, Austria) (ARES '22)*. Association for Computing Machinery, New York, NY, USA, Article 127, 10 pages. doi:10.1145/3538969.3544415
- [34] Marc Ohm, Henrik Plate, Arnold Sykosh, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings (Lisbon, Portugal)*. Springer-Verlag, Berlin, Heidelberg, 23–43. doi:10.1007/978-3-030-52683-2_2
- [35] ossilate inc. 2022. Packj flags malicious/risky open-source packages. <https://github.com/ossilate-inc/packj>. Accessed: 2025-4-16.
- [36] Phylum. 2024. NPM Security Holding. https://docs.phylum.io/analytics/npm_security_holding. Accessed: 2025-07-08.
- [37] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992. doi:10.18653/v1/D19-1410
- [38] Simone Scalco, Ranindya Paramitha, Duc-Ly Vu, and Fabio Massacci. 2022. On the feasibility of detecting injections in malicious npm packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security (Vienna, Austria) (ARES '22)*. Association for Computing Machinery, New York, NY, USA, Article 115, 8 pages. doi:10.1145/3538969.3543815
- [39] Securelist. 2022. LofyLife: malicious npm packages steal Discord tokens and bank card data. <https://securelist.com/loflylife-malicious-npm-packages/107014/>. Accessed: 2025-07-08.
- [40] Adriana Sejfa and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software*

- Engineering (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 1681–1692. doi:10.1145/3510003.3510104
- [41] Vikas Singh. 2025. 50+ Node.js Statistics Covering Usage, Adoption, and Performance. <https://www.brilworks.com/blog/nodejs-usage-statistics/>. Accessed: 2025-07-08.
- [42] Matthew Taylor, Raturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending Against Package Typosquatting. In *Network and System Security: 14th International Conference, NSS 2020, Melbourne, VIC, Australia, November 25–27, 2020, Proceedings* (Melbourne, VIC, Australia). Springer-Verlag, Berlin, Heidelberg, 112–131. doi:10.1007/978-3-030-65745-1_7
- [43] Tree-sitter. 2025. Tree-sitter: An incremental parsing system for programming tools. <https://github.com/tree-sitter/tree-sitter>. Accessed: 2025-07-08.
- [44] Ritwik Verma. 2026. Node.js Statistics: The Updated Guide on Node.js Usage and Trends. <https://www.bacancytechnology.com/blog/nodejs-statistics>. Accessed: 2026-03-10.
- [45] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad Snakes: Understanding and Improving Python Package Index Malware Scanning. In *Proceedings of the 45th International Conference on Software Engineering* (Melbourne, Victoria, Australia) (ICSE '23). IEEE Press, 499–511. doi:10.1109/ICSE48619.2023.00052
- [46] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Typosquatting and Combosquatting Attacks on the Python Ecosystem. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroSPW)*. 509–514. doi:10.1109/EuroSPW51379.2020.00074
- [47] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. 2025. MalPacDetector: An LLM-Based Malicious NPM Package Detector. *IEEE Transactions on Information Forensics and Security* 20 (2025), 6279–6291. doi:10.1109/TIFS.2025.3580336
- [48] Elizabeth Wyss, Dominic Tassio, Lorenzo De Carli, and Drew Davidson. 2025. Evaluating LLM-Based Detection of Malicious Package Updates in npm. In *2025 28th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. IEEE, 678–692. doi:10.1109/RAID67961.2025.00047
- [49] Zeliang Yu, Ming Wen, Xiaochen Guo, and Hai Jin. 2024. Maltracker: A Fine-Grained NPM Malware Tracker Copiloted by LLM-Enhanced Dataset. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 1759–1771. doi:10.1145/3650212.3680397
- [50] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. 2025. Leveraging Large Language Models to Detect npm Malicious Packages. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) (ICSE '25). IEEE Press, 2625–2637. doi:10.1109/ICSE55347.2025.00146
- [51] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice* (Pittsburgh, Pennsylvania) (ICSE-SEIP '22). Association for Computing Machinery, New York, NY, USA, 331–340. doi:10.1145/3510457.3513044
- [52] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruizi Wang, Chong Wang, and Xin Peng. 2025. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence. *ACM Trans. Softw. Eng. Methodol.* 34, 4, Article 104 (April 2025), 28 pages. doi:10.1145/3705304
- [53] Yu Zhang, Haipeng Qu, Lingyun Ying, and Linghui Wang. 2024. Maldet: An Automated Malicious npm Package Detector Based on Behavior Characteristics and Attack Vectors. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 1942–1947. doi:10.1109/TrustCom63139.2024.00269
- [54] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. 2024. Models Are Codes: Towards Measuring Malicious Code Poisoning Attacks on Pre-trained Model Hubs. In *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2087–2098. doi:10.1145/3691620.3695271
- [55] Xiaoyan Zhou, Ying Zhang, Wenjia Niu, Jiqiang Liu, Haining Wang, and Qiang Li. 2025. An Analysis of Malicious Packages in Open-Source Software in the Wild. In *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 371–385. doi:10.1109/DSN64029.2025.00045
- [56] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proceedings of the 28th USENIX Conference on Security Symposium* (Santa Clara, CA, USA) (SEC'19). USENIX Association, USA, 995–1010.